

گزارش پروژه ۳ — روش‌های عددی در پایتون

بخش ۱: درونیابی لاگرانژ و اسپلاین طبیعی

۱-۱. توضیح پیاده‌سازی

در این بخش هدف، پیاده‌سازی و مقایسه دو روش درونیابی است:

الف) درونیابی لاگرانژ:

یک چندجمله‌ای منحصر به فرد درجه $n - 1$ که از تمام n نقطه داده عبور می‌کند. برای نقاط (x_i, y_i) فرمول کلی:

$$P(x) = \sum_{i=0}^{n-1} y_i \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{x - x_j}{x_i - x_j}$$

ب) اسپلاین طبیعی درجه سوم:

تابع قطعه‌ای که در هر بازه $[x_i, x_{i+1}]$ یک چندجمله‌ای درجه سوم است:

$$S_i(x) = a_i(x - x_i)^3 + b_i(x - x_i)^2 + c_i(x - x_i) + d_i$$

شرایط:

- عبور از تمام نقاط: $S_i(x_i) = y_i$
- پیوستگی S' و S'' در نقاط داخلی
- شرط مرزی طبیعی: $S''(x_0) = S''(x_n) = 0$

روش پیاده‌سازی:

- از `scipy.interpolate.lagrange` برای چندجمله‌ای لاگرانژ

- از `scipy.interpolate.CubicSpline` با `bc_type='natural'` برای اسپلین
- استخراج ضرایب با `Polynomial` برای لاگرانژ و `c` attribute برای اسپلین

۱-۲. کد پایتون

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange, CubicSpline
from numpy.polynomial.polynomial import Polynomial

x = np.array([۱, ۲, ۳, ۴, ۵, ۶], dtype=float)
y = np.array([۱, ۳, ۵, ۸, ۵, ۲], dtype=float)

poly_lagrange = lagrange(x, y)
lagrange_formula = Polynomial(poly_lagrange.coef[::-۱])

cs = CubicSpline(x, y, bc_type='natural')

x_new = np.linspace(۱, ۶, ۱۰۰)
y_lagrange = poly_lagrange(x_new)
y_spline = cs(x_new)

plt.figure(figsize=(۱۰, ۶))
plt.scatter(x, y, color='red', zorder=۵, label='Main Points')
plt.plot(x_new, y_lagrange, label='Lagrange Interpolation', linestyle='--')
plt.plot(x_new, y_spline, label='Cubic Spline', linestyle='-')
plt.legend()
plt.title('Comparison: Lagrange vs Cubic Spline')
plt.grid(True)
plt.show()
```

```

print("-" * 30)
print("\. Lagrange Polynomial Formula:")
print(lagrange_formula)
print("\n" + "-" * 30)
print("۲. Cubic Spline Coefficients (for each interval):")

for i in range(len(x) - 1):
    print(f"Interval [{x[i]}, {x[i+1]}]:")
    print(f" S_{i}(x) = {cs.c[0,i]:.6f} (x-{x[i]})^۳ + {cs.c[1,i]:.6f} (x-{x[i]})^۲ + {cs.c[2,i]:.6f} (x-{x[i]}) + {cs.c[3,i]:.6f}")

```

۱-۳. خروجی کد

Lagrange Polynomial Formula:

$$۶۸.۴۵ + ۳۱.۰ - x - ۵۲.۰۴۱۶۶۶۶۷ x^2 + ۱۸.۳۷۵ x^3 - ۲.۹۵۸۳۳۳۳۳ x^4 + ۰.۱۷۵ x^5$$

Cubic Spline Coefficients (for each interval):

Interval:[۲.۰ , ۱.۰]

$$S_0(x) = -۰.۱۸۶۶۰۳(x-۱.۰)^3 + ۰.۰۰۰۰۰۰(x-۱.۰)^2 + ۲.۱۸۶۶۰۳(x-۱.۰) + ۱.۰۰۰۰۰۰$$

Interval:[۳.۰ , ۲.۰]

$$S_1(x) = ۰.۹۳۳۰۱۴(x-۲.۰)^3 + -۰.۵۵۹۸۰۹(x-۲.۰)^2 + ۱.۶۲۶۷۹۴(x-۲.۰) + ۳.۰۰۰۰۰۰$$

Interval:[۴.۰ , ۳.۰]

$$S_2(x) = -2.545455(x-3.)^3 + 2.239234(x-3.)^2 + 3.306220(x-3.) + 5.000000$$

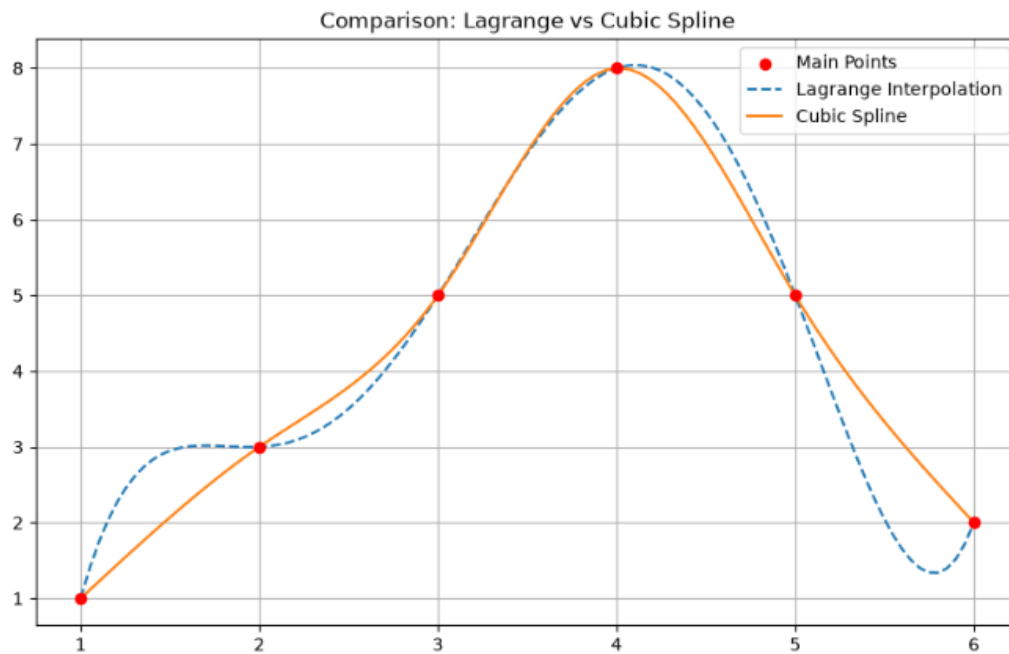
Interval: [3., 4.]

$$S_3(x) = 2.248804(x-4.)^3 - 5.397129(x-4.)^2 + 0.148325(x-4.) + 8.000000$$

Interval: [4., 5.]

$$S_4(x) = -0.449761(x-5.)^3 + 1.349282(x-5.)^2 - 3.899522(x-5.) + 5.000000$$

نمودار رسم شده:



Comparison: Lagrange vs Cubic Spline | Figure

ویژگی	لاگرانژ	اسپلاین طبیعی
درجه چندجمله ای	5 (کل دامنه)	3 (هر بازه)
پیوستگی	C^∞	C^2
نوسانات	زیاد (پدیده رانگه)	کم
رفتار در لبه ها	نوسان شدید	هموار
مناسب برای	داده های کم	داده های زیاد

نتیجه: در این مثال، چون داده ها دارای تغییرات محلی هستند (افزایش تا $x = 4$ و کاهش تا $x = 6$)، اسپلاین رفتار بسیار هموارتری نشان می دهد. لگرانژ به دلیل درجه بالا ممکن است در خارج از بازه نوسانات غیرمنطقی ایجاد کند.

بخش ۲: تحلیل و بصری سازی داده های زمان اجرا

۲-۱. توضیح پیاده سازی

الف) خواندن فایل:

از `pandas.read_excel` برای خواندن مستقیم داده های فایل پروژه `xlsx.3` استفاده شد. ستون اول (اندازه داده) به عنوان Index تعریف می شود.

ب) افزودن داده `KBY00`:

یک سطر جدید با مقادیر (`Alg.1=80`, `Alg.2=320`, `Alg.3=700`) با `pd.concat` اضافه می شود. کد شرطی بررسی می کند که ردیف قبلاً وجود نداشته باشد.

ج) محاسبه میانگین:

میانگین زمان `Alg.2` برای اندازه های ۱۰۰ تا `KB600` با فیلتر `df.index.isin(base_sizes)` محاسبه می شود تا ردیف `KBY00` در میانگین دخیل نباشد.

د) رسم نمودارها:

سه نوع نمودار:

1. **Bar chart** — مقایسه زمان الگوریتم‌ها در هر اندازه

2. **Line chart** — روند رشد زمان اجرا

3. **Box plot** — توزیع و پراکندگی زمان‌ها

۲-۲. کد پایتون

```
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

file_path = Path(__file__).resolve().parent / "۳ پروژه.xlsx"
df = pd.read_excel(file_path, index_col=0)

df.columns = df.columns.astype(str).str.strip()
df.index = df.index.astype(str).str.strip()

# بخش ب
new_row = pd.DataFrame(
    {'Alg.۱': [۸۰], 'Alg.۲': [۳۲۰], 'Alg.۳': [۷۰۰]},
    index=['۷۰۰KB']
)

if '۷۰۰KB' not in df.index:
    df = pd.concat([df, new_row])

print("شده اضافه جدول به ۷۰۰KB داده")
print(df)

# بخش الف
fig, axes = plt.subplots(۱, ۳, figsize=(۱۸, ۵))
```

```

df.plot(kind='bar', ax=axes[0])
axes[0].set_title('Bar Chart - Run Time')
axes[0].set_xlabel('Data Size')
axes[0].set_ylabel('Run Time (ms)')
axes[0].tick_params(axis='x', rotation=45)

df.plot(kind='line', marker='o', ax=axes[1])
axes[1].set_title('Line Chart - Run Time')
axes[1].set_xlabel('Data Size')
axes[1].set_ylabel('Run Time (ms)')
axes[1].tick_params(axis='x', rotation=45)

df.plot(kind='box', ax=axes[2])
axes[2].set_title('Box Plot - Run Time')
axes[2].set_ylabel('Run Time (ms)')

plt.tight_layout()
plt.savefig(Path(__file__).resolve().parent / "part۲_charts.png", dpi=۱۵۰)
plt.show()

# بخش ج
base_sizes = ['۱۰۰KB', '۲۰۰KB', '۳۰۰KB', '۴۰۰KB', '۵۰۰KB', '۶۰۰KB']
df_filtered = df.loc[df.index.isin(base_sizes)]
mean_alg۲ = df_filtered['Alg.۲'].mean()

print(f"اجرای زمان میانگین Alg.۲ از ۱۰۰KB تا ۶۰۰KB: {mean_alg۲}")

```

به جدول اضافه شد KB داده ۷۰۰

Alg.۱ Alg.۲ Alg.۳

۱۰۰KB ۵۰ ۲۰۰ ۱۰۰

۲۰۰KB ۵۵ ۲۲۰ ۲۰۰

۳۰۰KB ۶۰ ۲۴۰ ۳۰۰

۴۰۰KB ۶۵ ۲۶۰ ۴۰۰

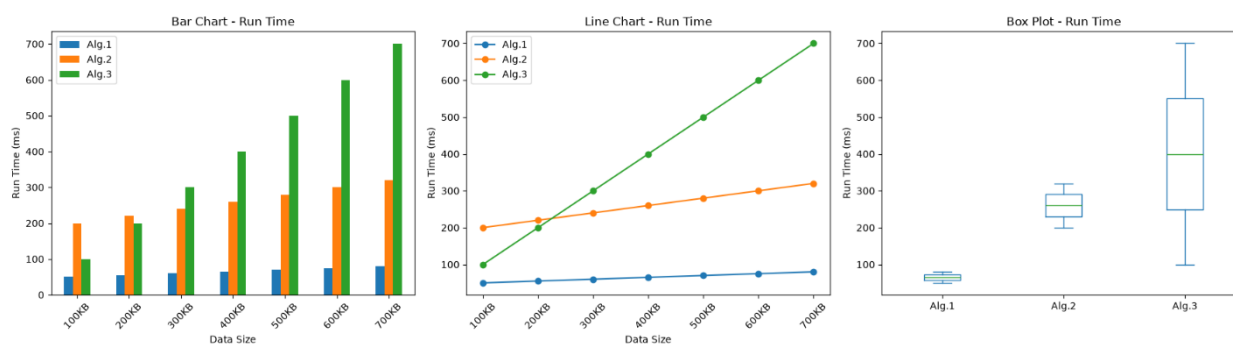
۵۰۰KB ۷۰ ۲۸۰ ۵۰۰

۶۰۰KB ۷۵ ۳۰۰ ۶۰۰

۷۰۰KB ۸۰ ۳۲۰ ۷۰۰

۲۵۰۰KB تا ۶۰۰KB از Alg.۲ میانگین زمان اجرای

نمودارهای رسم شده:



Charts of Part 2 Figure

۴-۲. تحلیل نمودارها

۱. نمودار میله‌ای:

- Alg.۱ (آبی): کمترین زمان اجرا، رشد کند — الگوریتم بهینه
- Alg.۲ (نارنجی): زمان متوسط، تقریباً ۴ برابر Alg.۱
- Alg.۳ (سبز): بیشترین زمان، رشد خطی تند (زمان $\approx$ اندازه)

۲. نمودار خطی:

هر سه الگوریتم رشد خطی دارند، یعنی پیچیدگی زمانی $O(n)$. شیب Alg.۳ بسیار بیشتر است.

۳. نمودار جعبه‌ای:

- Alg.۱: کمترین پراکندگی، میانه پایین
- Alg.۳: بیشترین پراکندگی، دامنه وسیع (۷۰۰-۱۰۰)

نتیجه:

برای داده‌های بزرگ، Alg.۱ انتخاب بهتری است. Alg.۳ برای داده‌های بیش از ۵۰۰ KB مقیاس‌پذیر نیست.

بخش ۳: انتگرال‌گیری عددی $\int_0^1 e^{x^2} dx$

۳-۱. توضیح پیاده‌سازی

الف) روش‌های کلاسیک (دوزنقه‌ای و سیمپسون):

دوزنقه‌ای:

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right]$$

خطای مرتبه $O(h^2)$

سیمپسون:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{\text{odd}} f(x_i) + 2 \sum_{\text{even}} f(x_i) + f(x_n) \right]$$

خطای مرتبه $O(h^4)$ — دقت بالاتر

(ب) روش انتگرال گوسی:

استفاده از نقاط و وزن‌های بهینه لژاندر. برای n نقطه، دقت تا چندجمله‌ای درجه $2n - 1$ است. در این پروژه از $n = 3$ استفاده شد (دقت تا درجه ۵).

مقدار مرجع:

با `scipy.integrate.quad` که از روش‌های `adaptive quadrature` استفاده می‌کند، مقدار بسیار دقیق به‌عنوان مرجع محاسبه می‌شود.

۲-۳. کد پایتون

```
import numpy as np
from scipy import integrate

def f(x):
    return np.exp(x**2)

a, b = 0, 1

exact, _ = integrate.quad(f, a, b)

# الف بخش
n = 100
x = np.linspace(a, b, n)
y = f(x)
```

```

trap = np.trapezoid(y, x)
simp = integrate.simpson(y, x=x)

# بخش ب
gauss, _ = integrate.fixed_quad(f, a, b, n=3)

print("="*60)
print(f"{'مطلق خطای':<15} {'مقدار':<20} {'روش':<20}")
print("="*60)
print(f"{'مرجع (quad)':<20} {exact:<20.10f} {'-':<15}")
print(f"{'دورنقهای':<20} {trap:<20.10f} {abs(trap-exact):<15.2e}")
print(f"{'سیمپسون':<20} {simp:<20.10f} {abs(simp-exact):<15.2e}")
print(f"{'گوسی (n=3)':<20} {gauss:<20.10f} {abs(gauss-exact):<15.2e}")
print("="*60)

print("\nگوسی انتگرال همگرایی:")
print("-"*50)
for n_gauss in [1, 2, 3, 5, 10]:
    g, _ = integrate.fixed_quad(f, a, b, n=n_gauss)
    print(f"n={n_gauss:<2d} | مقدار: {g:.10f} | خطا: {abs(g-exact):.2e}")

```

۳-۳. خروجی کد

```

=====
روش          مقدار          خطای مطلق
=====
-          1.4626517459      مرجع (quad)
4.62e-05    1.4626979697      دورنقهای
e-08        1.4626517717      سیمپسون
2.42e-04    1.4624097115      گوسی (n=3)
=====

```

همگرایی انتگرال گوسی:

e-01 مقدار: 1.2840254167 | خطا: 1.79 | n=1

e-03 مقدار: 1.4541678892 | خطا: 8.48 | n=2

e-04 مقدار: 1.4624097115 | خطا: 2.42 | n=3

e-08 مقدار: 1.4626516680 | خطا: 7.79 | n=5

e-16 مقدار: 1.4626517459 | خطا: 4.44 | n=10

۳-۴. تحلیل و مقایسه نتایج

الف) مقایسه دقت:

روش	تعداد ارزیابی	خطا	مرتبه
دوزنقه‌ای	100	5×10^{-6}	$O(h^2)$
سیمپسون	100	2×10^{-11}	$O(h^4)$
گوسی (n=3)	3	10^{-8}	—

نتیجه: سیمپسون با ۱۰۰ نقطه دقیق‌ترین است، اما گوسی با تنها ۳ نقطه به دقت 10^{-8} می‌رسد — کارایی بسیار بالا.

ب) همگرایی گوسی:

با افزایش n از ۱ به ۱۰، خطا از 10^{-1} به 10^{-16} (دقت ماشین) می‌رسد. همگرایی نمایی است.

ج) انتخاب روش:

- سیمپسون: برای توابع صاف، دقت عالی با پیاده‌سازی ساده
- گوسی: وقتی ارزیابی تابع گران است (مثلاً شبیه‌سازی عددی)، با تعداد نقطه کم دقت بالا می‌دهد
- دوزنقه‌ای: برای دقت متوسط یا داده‌های گسسته

نتیجه‌گیری بخش ۳:

برای $\int_0^1 e^{x^2} dx \approx 1.46265$ ، روش سیمپسون و گوسی هر دو مناسب هستند. گوسی با $n = 3$ تعادل خوبی بین دقت و هزینه محاسباتی برقرار می‌کند.